Spike: Task 8

Title: Game State Management

Author: Mohamed Shafi Uzman Fassy, 102608927

Goals / deliverables:

The goal of this spike task is to demonstrate my ability to collect and analyse software performance data while applying my knowledge in comparing and evaluating functions, IDE and complier settings.

Besides this report, what else was created?

- Code files in see C:\Users\storm\Documents\GitHub\GAMES PROGRAMMING REPO\COS30031-2023-102608927\08 - Spike - Game State Management\Zorkish\Zorkish

Technologies, Tools, and Resources used:

List of information needed by someone trying to reproduce this work.

- Visual Studio 2022 Community Edition
- GitHub
- Game Specification Details Canvas Module
- OO Concepts, Patterns & Structures Canvas Modules
    - 2.1 Software Architecture of Games Lecture Slides
    - 2.2 Game States and Stages Lecture Slides
    - 3.2 Data Structures
    - 3.3 Design Patterns

Tasks undertaken:

- Planning the Zorkish Adventure
- Download and install Visual Studio
- Download and install Visual Studio C++ Debugging Packages
- Create and Configure VS Project File with a .cpp file to build.

What we found out:

Planning Phase:

Based on the game specifications details and the requirements for phase 1 I designed this UML design that visualizes the Game State management and the structuring for the world and command processing.

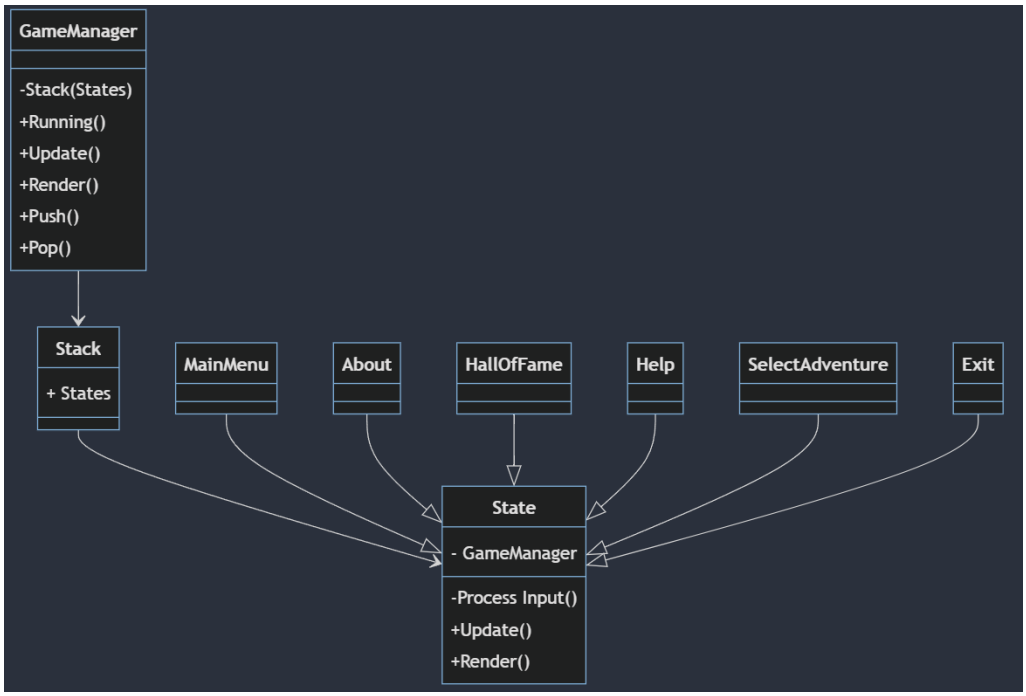

Figure 1 : Initial UML Design for Phase 1 of Zorkish Adventure Game

Developing State Class and StateManager:

```cpp
#pragma once
#include "GameManager.h"
#include <iostream>

using namespace std;

class GameManager;

class State
{
protected:
    GameManager *_manager = nullptr;
    char _command;

    virtual void processInput()
    {
        _command = cin.get();
        cin.clear();
        cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    };

public:
    virtual void update() = 0;
    virtual void render() = 0;
};
```

Based on my initial UML design the State header was created to estabilishes a clear methods that every state needs to render, update and get user input.

```h
#pragma once
#include "State.h"
#include "MainMenu.h"
#include <stack>

using namespace std;

class GameManager
{
private:
    stack<State*> _states;
public:
    GameManager();
    ~GameManager();

    bool running();
    State* current();

    void push_state(State*);
    void pop_state();
};
```

```cpp
#include "GameManager.h"

GameManager::GameManager()
{
    push_state(new MainMenu(this));
}

GameManager::~GameManager()
{
    while (!_states.empty())
        pop_state();
}

bool GameManager::running()
{
    return !_states.empty();
}

State* GameManager::current()
{
    return _states.top();
}

void GameManager::push_state(State* state)
{
    _states.push(state);
}

void GameManager::pop_state()
{
    if (!_states.empty())
    {
        delete _states.top();
        _states.pop();
    }
}
```

To manage these states, I created the GameManager that connects with the games main loop each state and therefore I use the currentstate to manage and store the current state of the game. GameManager handles state transitions as well with methods such as pushstate and popstate to make sure that one instance of GameManager exists at a time by using a Singleton pattern.

Developing Basic States:

When developing basic states such as Mainmenu, we use the header file for MainMenu to be the blueprint that inherits from State.h so does other states with their own header file that inherits from State.h as well as we want to make sure our code follows OOP design principles.

```cpp
#include "MainMenu.h"

MainMenu::MainMenu(GameManager* manager)
{
    _manager = manager;
}

MainMenu::~MainMenu()
{
    delete _manager;
    _manager = nullptr;
}

void MainMenu::update()
{
    processInput();

    switch (_command)
    {
    case '1':
        _manager->push_state(new SelectAdventure(_manager));
        break;
    case '2':
        _manager->push_state(new HallOfFame(_manager));
        break;
    case '3':
        _manager->push_state(new Help(_manager));
        break;
    case '4':
        _manager->push_state(new About(_manager));
        break;
    case '5':
        _manager->push_state(new Exit(_manager));
        break;
    default:
        cout << "Invalid input." << endl;
    }
}

void MainMenu::render()
{
    std::cout << "Zorkish :: Main Menu\n";
    std::cout << "-----------------------\n";
    std::cout << "1. Select Adventure and Play\n";
    std::cout << "2. Hall of Fame\n";
    std::cout << "3. Help\n";
    std::cout << "4. About\n";
    std::cout << "5. Quit\n";
    std::cout << "Select 1 - 5 :> ";
}

```

```cpp
#pragma once
#include "GameManager.h"
#include "State.h"
#include "About.h"
#include "HallOfFame.h"
#include "Help.h"
#include "SelectAdventure.h"
#include "Exit.h"
#include <iostream>

class MainMenu : public State
{
public:
    MainMenu(GameManager*);
    ~MainMenu();

    void update();
    void render();
};
```

The MainMenu class and all other states utilises basically the same methods to render and process basic user input. We use constructors to set the manager to pop and push between states while using update method to process the user commands using a switch case statement to find the next state to push in the stack. Moreover, render handles in printing out a visual Main Menu with display options for users to know what they should be entering to move between states.

Gameplay Phase 1 Functionality:

```cpp
#include "Gameplay.h"

void Gameplay::processInput()
{
    cin >> _command;
    cin.clear();
    cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

Gameplay::Gameplay(GameManager* manager)
{
    _manager = manager;
}

Gameplay::~Gameplay()
{
}

void Gameplay::update()
{
    processInput();

    if (_command == "quit")
    {
        _manager->pop_state();
    }
    else if (_command == "hiscore")
    {
        _manager->pop_state();
        _manager->push_state(new Highscore(_manager));
    }
    else
        cout << "Invalid input." << endl;
}

void Gameplay::render()
{
    std::cout << "Welcome to Zorkish :: Void World\n";
    std::cout << "-----------------------\n";
    std::cout << "This world is simple and pointless. Used it to test Zorkish phase 1 spec.\n";
    std::cout << ":> ";
}

```

Gameplay State is a bit different from other states as we process user inputs using string instead of int and this changes how the update method works as well. Currently results are hardcoded but in later phases we will be implementing more complex ways of displaying desired results.